Instrumenting P4 in the Linux kernel

Extending Linux networking with P4 and offloading to programmable HW

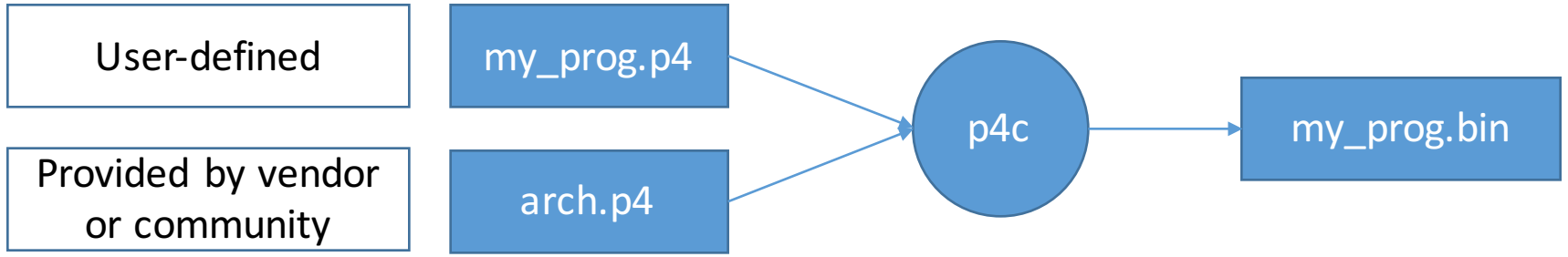
Antonin Bas, Ramkumar (Kram) Krishnamoorthy
Barefoot Networks

P4 & TC

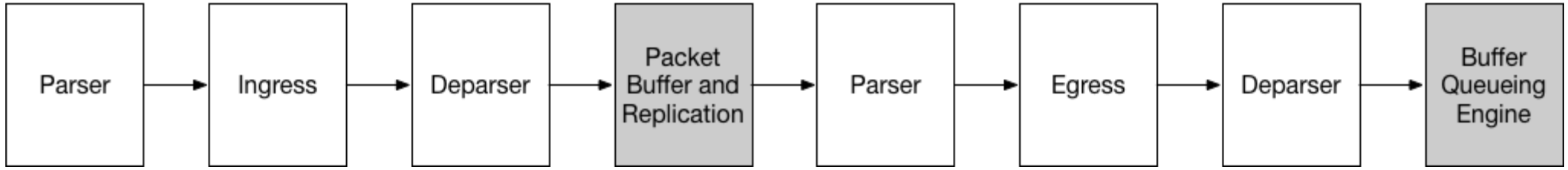
- Both use match-action processing (TC CA – P4 tables)
- P4 tables have fixed, compile-time list of actions
- P4 assumes a compiler, which implies a static datapath until the next recompilation. There is no incremental compilation and no dynamic reconfiguration of the match-action graphs through APIs.
  - Ensures P4 can be supported efficiently across many devices: ASICs / FPGAs
- P4 is very flexible in terms of programming the datapath, while TC is more runtime programming for an existing datapath
- P4 is very good at describing packet-processing in one place and generating intuitive & consistent control-plane APIs.

P4 architectures

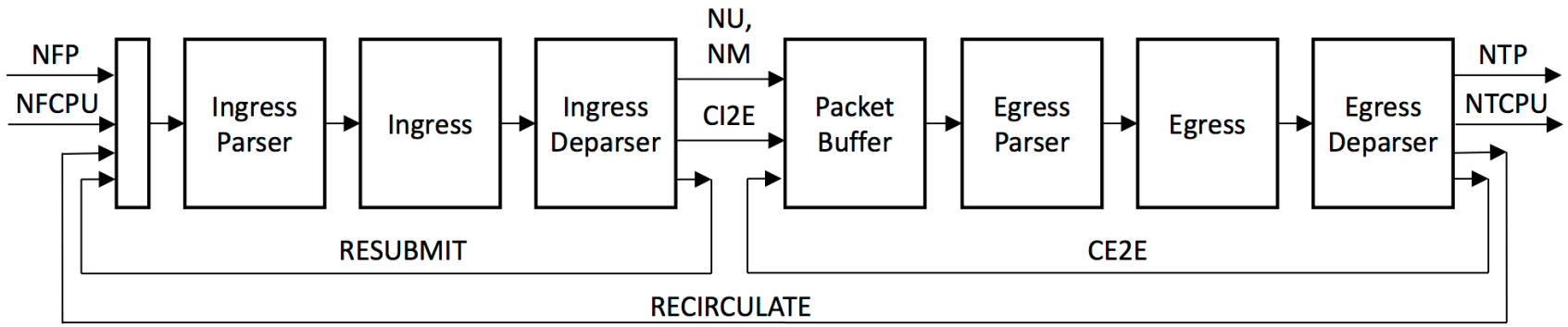
- Architecture = pipeline description + externs
- P4_16 introduces the notion of architecture
  - The language is no longer limited to an arbitrary switch model
  - Data-center class switching ASICs are no longer the only de-facto targets
  - Vendors can provide architecture definitions tailored to their HW for more efficient programming
  - Portability can still be achieved through “standard architectures”: Portable Switch Architecture (PSA), Portable NIC Architecture ?



PSA architecture = pipeline description + externs



PSA pipeline



PSA packet paths

PSA externs: counter (stats) + trTCM meter + stateful register + …

P4 can be different things to different people

- P4 native hardware vendors & OEMs: fine-grained way of programming the dataplane through target-specific native architectures
- SDN operators: a common abstraction across all programmable (and sometimes fixed-function) hardware which enables vendor-independence and next-gen SDN APIs such as P4Runtime
- **Efficient HW programming vs Portability & common abstractions**
- P4 native HW can often support more generic P4 architectures, but:
  - There may be some HW-driven limitations if the architecture includes some constructs which are *too* generic
  - More manual work every time in the compiler backend
- What can P4 bring to Linux:
  - describe existing Linux networking
  - extend Linux networking (with P4 runtime control API) in a uniform manner, independently of which forwarding engine is used
  - support native P4 HW in Linux

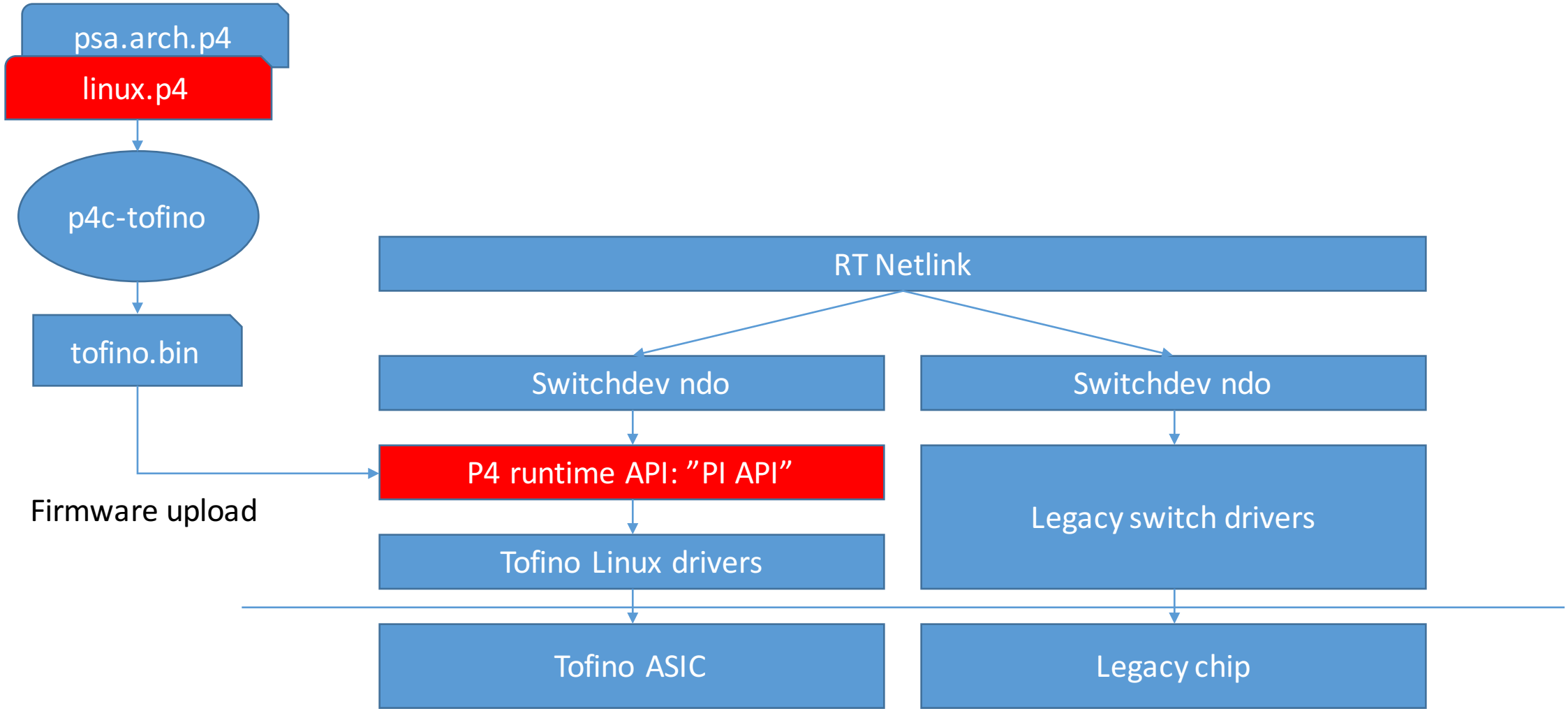
P4 networking

- What's easy to do – but not ideal:
  - Introduce new P4 architectures and write specialized P4 programs for every component: XDP, TC cls_bpf, SmartNIC, FPGA, ASIC
  - Control these components independently through their own respective APIs (eBPF maps, ASIC SDK, …) or through a P4 user-space API (e.g. P4Runtime)
  - Maybe this is a good way of building networks and it doesn't limit P4 programmable HW
  - But this can mean bypassing the Linux kernel for P4 native HW!
- What's hard to do – but we think is the right way to go:
  - A unified P4 program which runs in Linux and can be seamlessly offloaded to programmable hardware when available

Basic case: implement switchdev in P4

- Implement switchdev model in P4 / PSA for a data-center class switch (leaf / spine): fixed P4 pipeline
- Implement ndo functions on top of generic P4 drivers (PI API?) and upstream it
- Vendors which support P4 implement generic P4 drivers
- Pros:
  - Linux offload for P4 native HW (vendors just need to implement the generic P4 drivers), nothing changes for “legacy” devices
  - Useful description of Linux networking in P4?
- Cons:
  - **No user programmability**: adding new features require updating P4 model & switchdev ndo ops upstream

Switchdev in P4

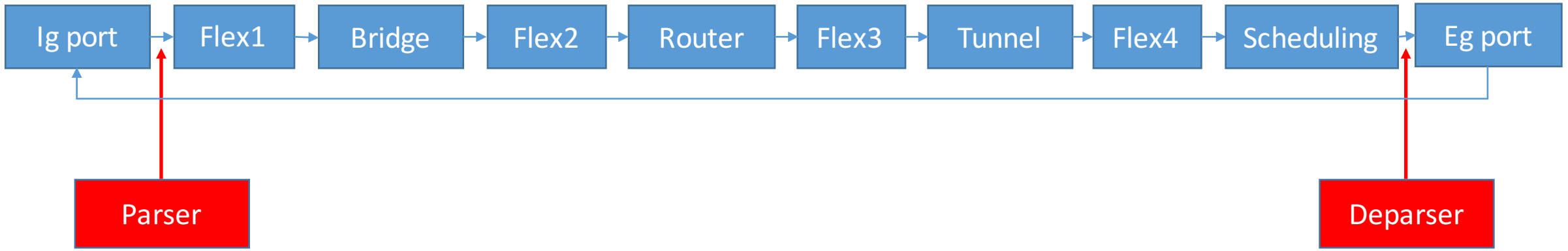


The PI API

- Designed specifically for runtime control of P4 objects (match-action tables and standard externs)
- Program-independent (works with any P4 program)
- Also used to configure the target with a new P4 dataplane (new firmware upload)
  - The binary is different for every compiler backend / for every target
- PI comes with a binary serialization format which could be used with generic NETLINK
- P4-programmable hardware (e.g. Barefoot Tofino ASIC) implements the PI API natively

How to instrument P4 programmability?

- Matty Kadosh @ Netdev 2.2: a P4 Linux architecture with flex blocks
- Top-down approach: uniform architecture designed specifically for the use-case (Linux offload) and not for HW
- Use various hooks in the kernel to run P4 custom code: fixed number of hooks at given point in the pipeline: exact pipeline TBD
- Can start with simple feature set and grow it



P4 Linux architecture

```
// linux.arch.p4

parser Parser<H, M>(inout H hdr, inout M user_meta);
control Deparser<H, M>(out H hdr, inut M user_meta);
control Flex1<H, M>(inout H hdr, inout M user_meta,
                    in linux_flex1_in_metadata_t flex1_in_meta,
                    inout linux_flex1_out_metadata_t flex1_out_meta);
control Flex2<H, M>(inout H hdr, inout M user_meta,
                    in linux_flex2_in_metadata_t flex2_in_meta,
                    inout linux_flex2_out_metadata_t flex2_out_meta);
//…
package Linux_Switch<H, M>(Parser<H, M>, Deparser<H, M>, Flex1<H, M>, Flex2<H, M>, …);

// input / output of flex blocks
struct linux_flex1_in_metadata_t { PortId_t ig_port; bit<12> vlan_tag; }
struct linux_flex1_out_metadata_t { bool drop_flag; bool fwd_flag; bit<32> nhop_idx; }
struct linux_flex2_in_metadata_t { PortId_t ig_port; bit<32> bridge_nhop_idx; }
struct linux_flex2_out_metadata_t { bool drop_flag; }

// actions: some standard actions that can be expressed in P4
// externs: “blackboxes” which cannot be expressed in P4 (some TC actions?)
```

Compiling flex blocks

- Flex blocks need to be compiled for SW + HW
  - SW: TC classifiers (cls_bpf, cls_u32, cls_flower?) or XDP (flex1 only)
  - HW: vendor-specific output for native P4 HW, existing ndo offload for TC classifiers and XDP for “legacy” devices
- Each vendor of native P4 hardware to provide a compiler backend that understands linux.arch.p4
- Need for a Linux compiler “driver” that can compile flex blocks to run into kernel SW (“driver” = p4c-xdp + p4c-tc + ?)
  - p4c-xdp: https://github.com/vmware/p4c-xdp
- Annotations can be used in P4 program to choose classifier: at the control level, at the table level or even at the block statement level

Annotations to choose classifier

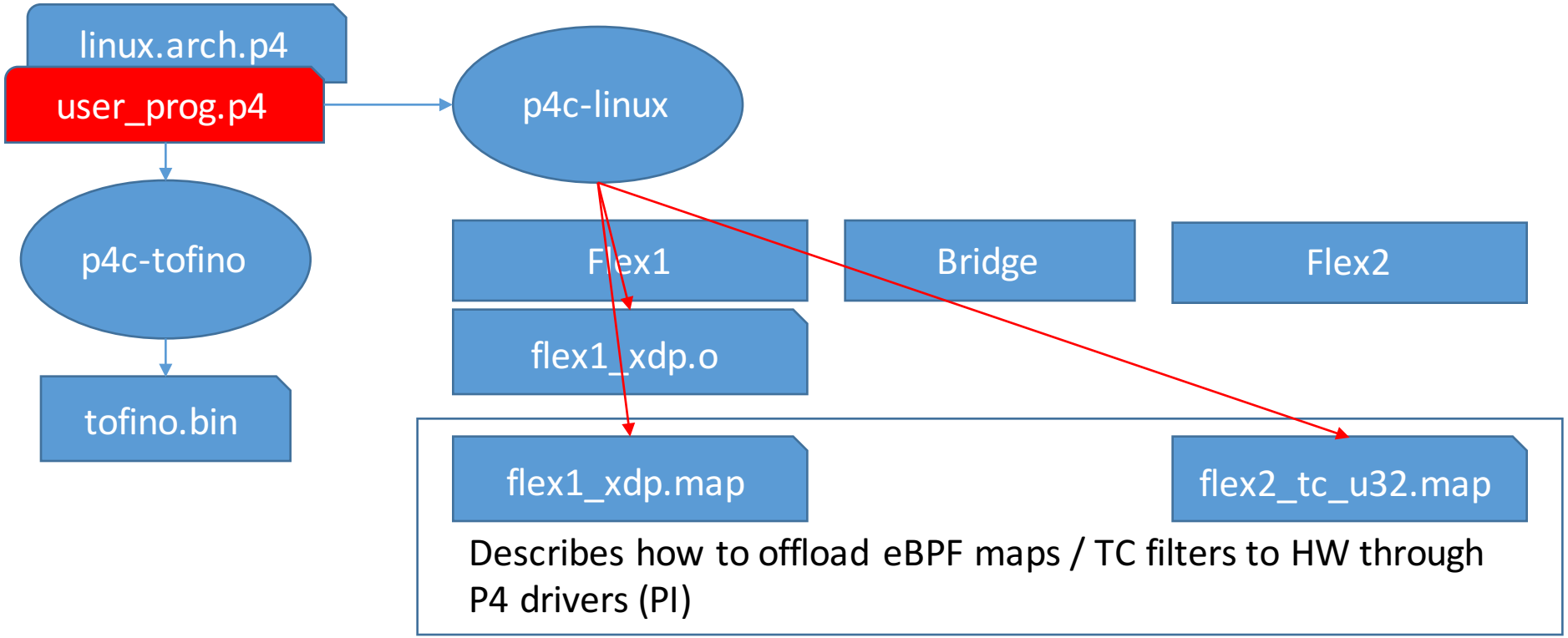
```
#include <linux.arch.p4>

parser MyParser(…) { … }

@linux_classifier(“xdp”)
control MyFlex1(…) { … }
// or
control MyFlex1(…) {
   table storm_control { … }
   table t2 { … }
   table t3 { … }
   table t4 { … }
  apply {
    @linux_classifier(“xdp”) t1.apply();
    @linux_classifier(“cls_bpf”) { t2.apply(); t3.appy(); }  // valid P4?
    @linux_classifier(“cls_u32”) t4.apply();
  }
}
```

P4 compiler rejects program if code cannot be realized with given classifier

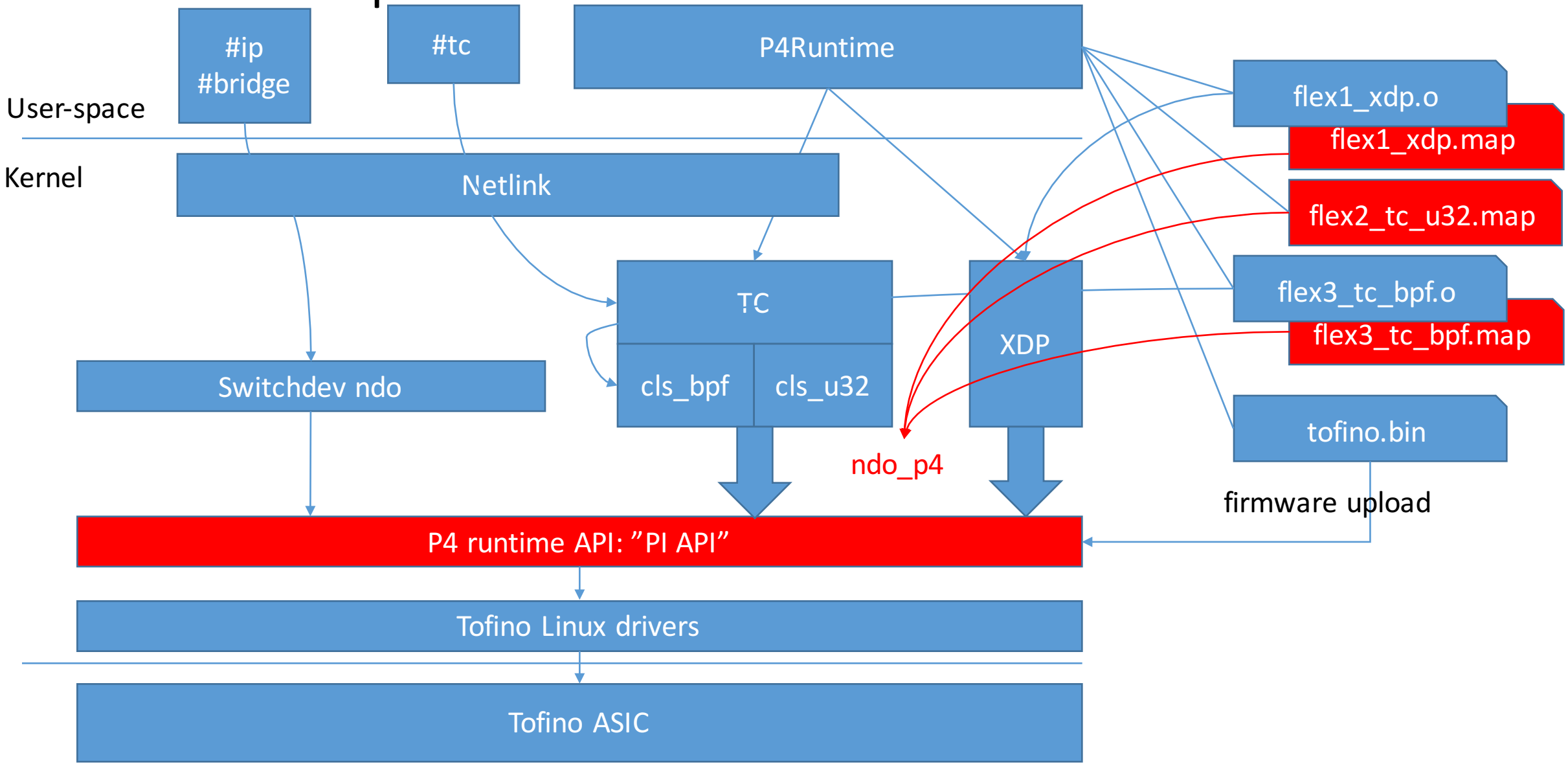
Compiling flex blocks



Control-path

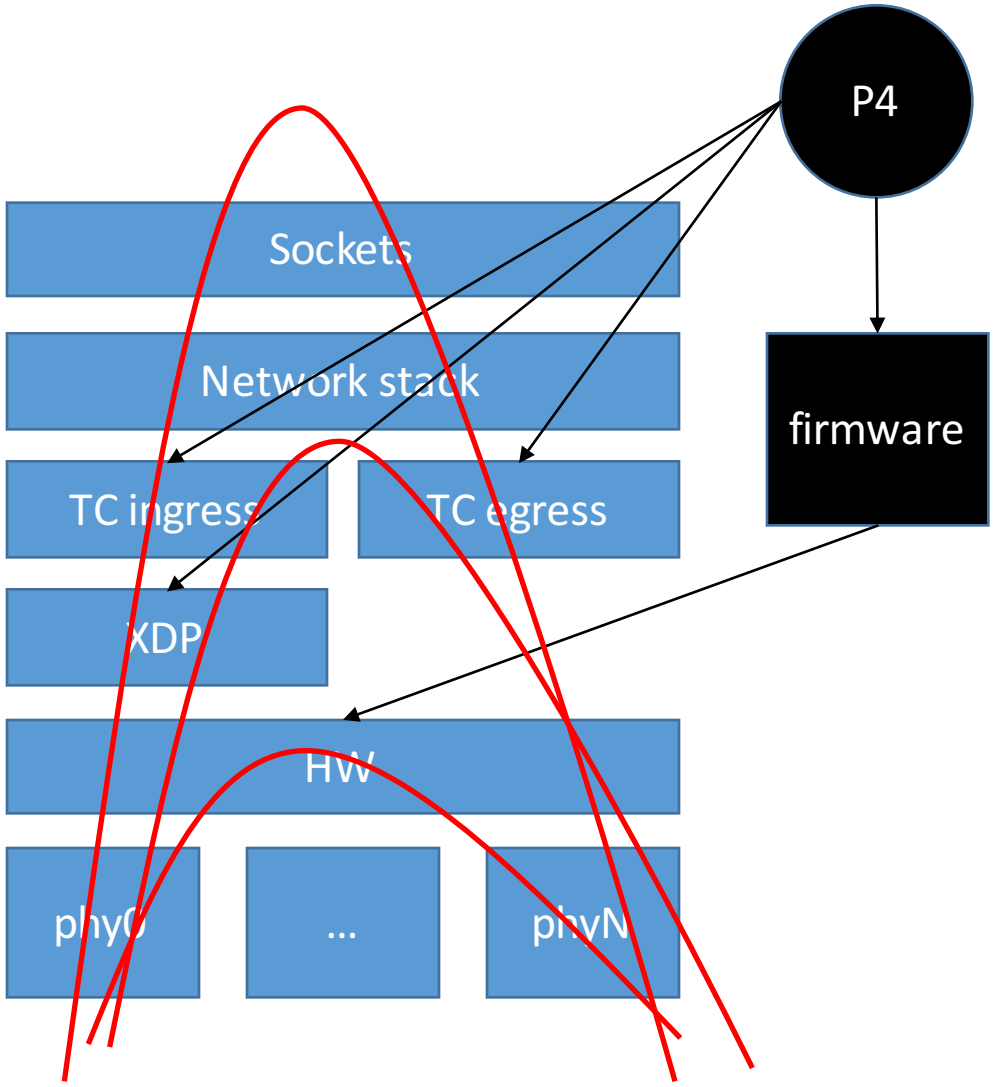
- User-space interface for flex blocks: P4Runtime?
- PI as generic driver for P4 native devices
- Offload of ”fixed-pipeline” (bridge, router) through switchdev
- TC classifier & XDP offload?
  - Existing ndo for legacy devices which already supports offload
  - New ndo ops for native P4 devices
    - Need to teach the kernel how to offload eBPF maps and TC filters through PI drivers
    - “Mapping” data published by the compiler

Control-path for P4 native device



Recap

- Leverages existing Linux pipeline and introduces some programmability through flex blocks
- Uses TC & XDP to realize flex blocks in SW
- HW offload through PI (native P4 HW) or existing offload mechanisms, with full SW fallback
- No generated kernel modules for packet processing
- Rocker to emulate native P4 device?



The case for kernel bypass

- Some P4 features may not be supported by TC / XDP, but may be supported by the HW, e.g. L4 heavy-hitter detection using Tofino stateful memories
- For these cases, we could consider a kernel bypass, with no software fallback
- Define new NETLINK family for direct access to PI drivers
  - Can use the existing binary serialization format for PI messages